Codeforces Round 1132 (Div. 2)

A1. 15K4JR3B (Easy Version)

1 second, 256 megabytes

— Did you know if you connect Park Kultury, a point not in the northeast, and one more on a map, you'll look like an idiot?
— No, I did not know that. Is that true?

This is the easy version of the problem. The difference between the versions is that in this version, in one operation, the coordinate of a point changes by exactly 1.

Unemployed ∗ Angel places three points A, B, and C at the origin.

You must perform exactly n operations of the following type: choose one of these points and increase or decrease one of its coordinates by 1. In other words, if the chosen point has coordinates (x, y), in one operation you may move it to $(x + 1, y)$, $(x - 1, y)$, $(x, y + 1)$, or $(x, y - 1)$.

Unemployed ∗ Angel asks you to find the maximum **doubled** area of triangle ABC after all operations are performed. As she is your idol, you cannot refuse her.

Since Unemployed ∗ Angel is very smart, she can show that the value of interest is always an integer.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 1000$). The description of the test cases follows.

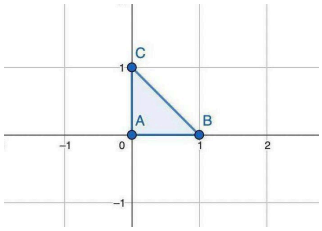
The only line of each test case contains one integer n ($2 \le n \le 1000$) — the number of operations.

Output

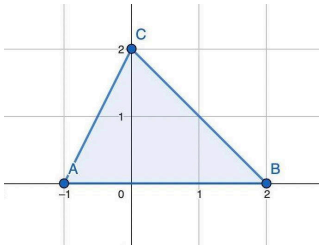
For each test case, output one integer — the maximum doubled area of triangle ABC.

input
5 2 5 15 43 769
output
1 6 56 462 147840

In the first test case, increase the x-coordinate of B by 1, then increase the y-coordinate of C by 1. After these 2 operations, the points are $A = (0, 0)$, $B = (1, 0)$, and $C = (0, 1)$. The doubled area of the resulting triangle is 1.



In the second test case, decrease the x-coordinate of A by 1, then increase the x-coordinate of B by 1 twice, and finally increase the y-coordinate of C by 1 twice. After these 5 operations, the points are $A = (-1, 0)$, $B = (2, 0)$, and $C = (0, 2)$. The doubled area of the resulting triangle is 6.



A2. 15K4JR3B (Hard Version)

1 second, 256 megabytes

— Did you know if you connect Park Kultury, a point not in the northeast, and one more on a map, you'll look like an idiot?
— No, I did not know that. Is that true?

This is the hard version of the problem. The difference between the versions is that in this version, in one operation, a point coordinate changes by a power of two.

Unemployed ∗ Angel places three points A, B, and C at the origin, and also gives you an array a of length n.

You must perform exactly n operations. During the i-th operation, choose one of these points and increase or decrease one of its coordinates by 2^{a_i}. In other words, if the chosen point has coordinates (x, y), then in the i-th operation you may move it to $(x + 2^{a_i}, y)$, $(x - 2^{a_i}, y)$, $(x, y + 2^{a_i})$, or $(x, y - 2^{a_i})$.

Unemployed ∗ Angel asks you to find the maximum **doubled** area of triangle ABC after all operations are performed. As she is your idol, you cannot refuse her.

Since Unemployed ∗ Angel is very smart, she can show that the value of interest is always an integer.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 100$). The description of the test cases follows.

The first line of each test case contains one integer n ($2 \le n \le 100$) — the length of array a and the number of operations.

The second line of each test case contains n integers $a_1, a_2, \ldots, a_n$ ($0 \le a_i \le 24$) — the elements of array a.

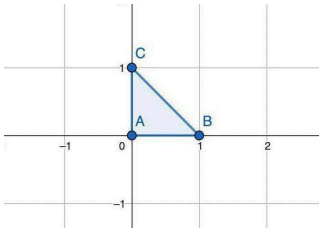
Output

For each test case, output one integer — the maximum doubled area of triangle ABC.

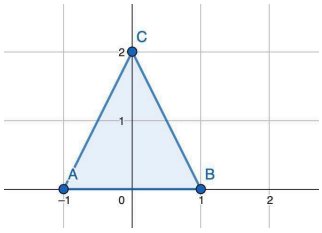
input
4 2 0 0 3 0 0 1 6 0 1 2 2 4 0 6 24 24 24 0 0 0

output
1 4 192 562950054084608

In the first test case, increase the x-coordinate of B by 2^0, then increase the y-coordinate of C by 2^0. After these 2 operations, the points are $A = (0, 0)$, $B = (1, 0)$, and $C = (0, 1)$. The doubled area of the resulting triangle is 1.



In the second test case, decrease the x-coordinate of A by 2^0, then increase the x-coordinate of B by 2^0, and finally increase the y-coordinate of C by 2^1. After these 3 operations, the points are $A = (-1, 0)$, $B = (1, 0)$, and $C = (0, 2)$. The doubled area of the resulting triangle is 4.



B. Introduction to the Niche Internet

1.5 seconds, 256 megabytes

Having failed to become a grandmaster in two months, Nikolai was completely disappointed in a career in competitive programming and decided to switch to political streaming.

It's known that no streamer can exist without the support of their beloved viewers, so every stream Nikolai necessarily receives some number of donations, forming a sequence a. Note that Nikolai is watched by a large number of hackers, so some donations may have negative values.

Nikolai considers the m-th donation to be *very pleasant* if its size is at least the average of all previous donations, that is, $a_m \cdot (m-1) \ge \sum_{i=1}^{m-1} a_i$. In particular, the first donation is always *very pleasant*.

Nikolai considers the stream to have paid off if during the broadcast he receives **exactly** k *very pleasant* donations.

Nikolai knows that today he will receive n donations from viewers, but he does not know in which order. Please help the viewers choose the order of sending donations so as to please Nikolai and make the stream pay off!

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 10^4$). The description of the test cases follows.

The first line of each test case contains two integers n and k ($1 \le k \le n \le 2 \cdot 10^5$) — the number of donations and the required number of *very pleasant* donations.

The second line of each test case contains n integers $a_1, a_2, \ldots, a_n$ ($-10^9 \le a_i \le 10^9$) — the donation sizes.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output "NO" if there is no suitable order of sending the donations.

Otherwise, output "YES" on the first line, and on the second line output n integers: the donation sizes in the order they are sent. If there are multiple solutions, output any of them.

You can output the answer in any case (upper or lower). For example, the strings "yEs", "yes", "Yes", and "YES" will be recognized as positive responses.

input
4 3 1 1 2 3 3 1 5 5 5 3 2 -1 -2 -3 4 3 0 0 10 -10

output
YES 3 2 1 NO YES -1 -3 -2 YES 10 -10 0 0

In the first test case, one suitable order is $[3, 2, 1]$. The first donation is always *very pleasant*. The second is not *very pleasant*, since $2 \cdot 1 < 3$, and the third is not, since $1 \cdot 2 < 3 + 2$. Thus, exactly 1 donation is *very pleasant*.

In the second test case, it can be shown that no suitable order exists.

In the third test case, one suitable order is $[-1, -3, -2]$. The first donation is *very pleasant*. The second is not *very pleasant*, since $(-3) \cdot 1 < -1$, while the third is, since $(-2) \cdot 2 \ge -1 + (-3)$. Thus, exactly 2 donations are *very pleasant*.

C1. Disgusting + GCD (Constructive Version)

2 seconds, 256 megabytes

This is the constructive version of the problem.

In this version, you are kijjot.

An array b is called *disgusting* if there exist indices $i < j$ such that $1 < \gcd(b_i, b_j) < \min(b_i, b_j)^*$.

You are given an array a of length n, consisting of positive integers. To prevent eternal merging, you need to make it **not** *disgusting*.

To do this, you can perform the following operation: choose an index $1 \le i \le n$ and a positive integer $1 \le x \le 10^9$ such that $\gcd(a_i, x) = 1$, and increase a_i by x. Note that the condition is checked using the **current** value of a_i before the operation, not its value in the original array.

Your strength is running low, so you can perform operations subject to the following restrictions:

- You can perform at most $n + 10$ operations.
- The sum of the values of x over all performed operations must not exceed the sum of the elements of the **original** array a.

Find any valid array obtained as a result of applying the operations. It can be shown that under the given constraints, such an array always exists.

*$\gcd(x, y)$ denotes the greatest common divisor (GCD) of integers x and y.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 10^4$). The description of the test cases follows.

The first line of each test case contains a single integer n ($2 \le n \le 10^5$) — the length of the array.

The second line contains n integers $a_1, a_2, \ldots, a_n$ ($2 \le a_i \le 10^5$) — the elements of the array.

It is guaranteed that the sum of n over all test cases does not exceed 10^5.

Output

For each test case, output n integers $b_1, b_2, \ldots, b_n$ ($a_i \le b_i \le 10^9$) — the array b resulting from the operations.

The array b must not be *disgusting* and must be obtainable from a using the allowed operations while satisfying both restrictions. If there are multiple valid arrays, output any of them.

input
3 2 7 12 3 7 9 12 7 8 76 49 26 24 80 99

output
7 12 7 25 12 9 77 50 27 25 81 100

In the first example, we can perform no operations, since the array is already not *disgusting*.

In the second example, the array is *disgusting* because $1 < \gcd(a_2, a_3) = \gcd(9, 12) = 3 < \min(a_2, a_3) = \min(9, 12) = 9$ holds. We can choose $i = 2$ and $x = 16$, since $\gcd(x, a_i) = \gcd(16, 9) = 1$. After this, the array a will become equal to $[7, 25, 12]$ and will no longer be *disgusting*.

C2. Disgusting + GCD (Checker Version)

2 seconds, 256 megabytes

In this version, you need to implement part of the checker for the constructive version.

In this version, you are ogiyo.

kijjot gives you two integers a and b such that $a \le b$. In one operation, you can choose a positive integer x such that $\gcd(a, x) = 1^*$, and then increase a by x.

Your task is to find any sequence of operations of the minimum possible length to transform a into b. It can be shown that under the given constraints, it is always possible to transform a into b using at most 15 operations.

*$\gcd(x, y)$ denotes the greatest common divisor (GCD) of integers x and y.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 10^4$). The description of the test cases follows.

Each test case contains two integers a and b ($2 \le a \le 10^5$, $a \le b \le 10^9$) — the numbers given to you by kijjot.

Output

For each test case, output the minimum number of operations k ($0 \le k \le 15$) in the first line.

In the second line, output k positive integers $x_1, x_2, \ldots, x_k$ — the values of x chosen in the operations, in order. These operations must be valid and transform a into b.

If $k = 0$, you may omit the second line. If there are multiple optimal sequences, output any of them.

input
4 2 2 15 25 2184 2200 100000 1000000000

output
0 2 1 9 3 11 2 3 2 1636673 998263327

In the first test case, $a = b = 2$, so no operations are needed.

In the second test case, we can first add 1 to obtain 16, and then add 9 to obtain 25. Both operations are valid because $\gcd(15, 1) = 1$ and $\gcd(16, 9) = 1$. It can be shown that fewer than 2 operations are not enough.

D. The Theorem of Two Cans

2 seconds, 256 megabytes

This is a run-twice (communication) problem.

The Left Can and the Right Can are catching a dangerous criminal hiding in the streets of Iron City. However, since they can only sing AND deny what is happening, they also need someone who is good at XOR.

The Prefect of the Unit's Bureau (PUB) is carrying out an important mission to train his agents, LCB and RCB. For this purpose, he will run a series of tests with them. Each test consists of two stages. During a test, the agents cannot communicate directly with each other.

In the first stage, LCB receives an integer n and a binary string s of length $n-1$. LCB must output a **connected** undirected graph on vertices $1, 2, \ldots, n$, without self-loops or multiple edges.

After receiving the graph from LCB, PUB may apply the following operation to it any number of times (possibly zero):

- choose an integer k ($k \geq 2$) and a simple* path $v_1, v_2, \ldots, v_{2k}$, consisting of $2k$ vertices and $2k-1$ edges;
- choose any index i ($1 \leq i \leq 2k-1$) and remove the edge between vertices v_i and v_{i+1} from the graph;
- if there is no edge between vertices v_1 and v_{2k}, add it.

Note that the number of edges may decrease after an operation. It can be shown that the graph remains connected.

Before passing the graph to RCB, PUB may arbitrarily reorder the edges and swap the two endpoints of any edge. This is possible even if PUB performs no operations.

In the second stage, PUB gives the resulting graph to RCB. RCB must recover the original string s. Find a strategy for both agents that succeeds regardless of the operations performed by PUB.

Your program will be run twice on each test: first as LCB, then as RCB. These are separate executions; the program's memory is not preserved between them.

First Run

Input

The first line contains the string `first`, indicating that this is the first run.

After this line, the remaining input has the following format.

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 100$). The description of the test cases follows.

The first line of each test case contains one integer n ($3 \leq n \leq 10^3$) — the number of vertices.

The second line contains a string s of length $n-1$, consisting only of the characters `0` and `1`.

It is guaranteed that the sum of n over all test cases does not exceed 10^3.

Output

For each test case, output an integer m ($n-1 \leq m \leq \frac{n(n-1)}{2}$) — the number of edges in your graph. In each of the next m lines, output two integers u and v ($1 \leq u, v \leq n$, $u \neq v$), describing an edge.

The graph must be connected and contain no self-loops or multiple edges. You may output the edges and their endpoints in any order.

Second Run

Input

The first line contains the string `second`, indicating that this is the second run.

After this line, the remaining input has the following format.

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 100$). The description of the test cases follows.

The value of t is the same as on the first run, although the test cases may appear in a different order.

The first line of each test case contains two integers n and m ($3 \leq n \leq 10^3$; $n-1 \leq m \leq \frac{n(n-1)}{2}$) — the number of vertices and edges in the resulting graph. The value of n is the same as in the corresponding test case of the first run.

Each of the next m lines contains two integers u and v ($1 \leq u, v \leq n$, $u \neq v$), describing an undirected edge.

The edges may be listed in any order, and the two endpoints of each edge may be given in either order, even if PUB performs no operations. These orders need not match your output in the first run.

It is guaranteed that the graph is obtained from your graph in the first run by applying the operations described above zero or more times. It can be shown that the resulting graph is connected and contains no self-loops or multiple edges.

Output

For each test case, output the original binary string s of length $n-1$.

*A path is called simple if all its vertices are distinct.

input

```
first
2
4
101
3
11
```

output

```
5
1 2
2 3
1 3
4 1
2 4
3
1 2
1 3
2 3
```

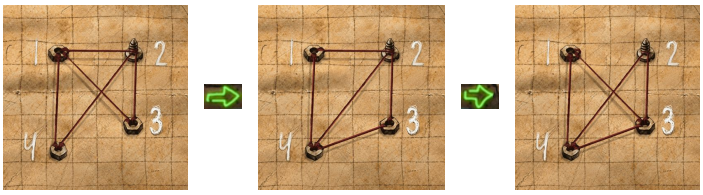
input

```
second
2
4 5
4 3
2 3
4 1
1 3
2 4
3 3
2 3
3 1
2 1
```

output

```
101
11
```

The example shows both runs: first the input and output of the first run, then the second.



In the first test case, $n = 4$ and $s = \texttt{101}$. LCB returned a graph with edges $(1, 2)$, $(2, 3)$, $(1, 3)$, $(4, 1)$ and $(2, 4)$. PUB chose $k = 2$ and the path $4 - 2 - 1 - 3$, removed edge $(1, 3)$, and added edge $(4, 3)$, and then chose $k = 2$ and the path $1 - 2 - 4 - 3$, removed edge $(1, 2)$, and added edge $(1, 3)$. RCB received the graph with edges $(4, 3)$, $(2, 3)$, $(4, 1)$, $(1, 3)$, $(2, 4)$ and reconstructed the string $\texttt{101}$.

In the second test case, PUB performs no operations. The graph is unchanged, but its edges are listed as $(2, 3)$, $(3, 1)$, and $(2, 1)$: their order and the order of some endpoints differ from the first run. RCB recovers $\texttt{11}$.

E. My Mind Is Going

2 seconds, 256 megabytes

This is an interactive problem.

Out of boredom, you decided to pass the time by playing a game of your own invention with the onboard supercomputer. The game works as follows: the supercomputer shows you a bipartite graph G on n vertices, and then chooses a secret pair of distinct edges e_1 and e_2 that are absent from it and have the following properties:

- the graph $G \cup \{e_1\}$ is bipartite;
- the graph $G \cup \{e_2\}$ is bipartite;
- the graph $G' = G \cup \{e_1, e_2\}$ is not bipartite.

Your task is to guess **at least one** of the secret edges. To do this, you may use two types of queries:

- `? 1` k v_1 v_2 $\ldots$ v_k ($1 \le k \le n$, all v_i are distinct): choose a set of k vertices and find out whether the subgraph of G', consisting only of these vertices and the edges of G' connecting them, is bipartite;
- `? 2` u v ($1 \le u, v \le n, u \ne v$): find out whether G' contains the edge (u, v).

It is guaranteed that graph G is such that the supercomputer can choose a pair of edges with the properties described above.

Since the manufacturer of supercomputers of your model has introduced new quotas on RAM usage, you may make no more than 33 queries.

Note that the interactor is non-adaptive: the hidden edges e_1 and e_2 are fixed before the interaction begins and do not depend on your queries.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 10^4$). The description of the test cases follows.

The first line of each test case contains two integers n and m ($4 \le n \le 2000$, $1 \le m \le \min\left(2 \cdot 10^5, \frac{n(n-1)}{2}\right)$) — the number of vertices and edges in graph G.

Each of the next m lines contains two integers u and v ($1 \le u, v \le n$, $u \ne v$) — an edge of graph G.

It is guaranteed that graph G is bipartite, contains no multiple edges, and that there exists a pair of edges e_1 and e_2 with the properties described above.

It is guaranteed that the sum of n^2 over all test cases does not exceed $4 \cdot 10^6$.

It is guaranteed that the sum of m over all test cases does not exceed $2 \cdot 10^5$.

Interaction

For each test case, first read graph G in the format described above, after which you may make no more than 33 queries of two types:

- `? 1` k v_1 v_2 $\ldots$ v_k ($1 \le k \le n$, all v_i are distinct). In response, you will receive `1` if the subgraph of G', consisting only of the specified vertices and the edges of G' connecting them, is bipartite, and `0` otherwise.
- `? 2` u v ($1 \le u, v \le n, u \ne v$). In response, you will receive `1` if the edge (u, v) is present in graph G', and `0` otherwise.

To report your answer, output one of the hidden edges in the format `!` u v. After that, immediately proceed to the next test case. Outputting the answer does not count as a query.

After printing each query do not forget to output the end of line and flush* the output. Otherwise, you will get `Idleness limit exceeded` verdict.

If, at any interaction step, you read -1 instead of valid data, your solution must exit immediately. This means that your solution will receive `Wrong answer` because of an invalid query or any other mistake. Failing to exit can result in an arbitrary verdict because your solution will continue to read from a closed stream.

Hack format

The first line contains one integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains two integers n and m, the next m lines contain the edges of graph G in the format described above. Each of the next two lines contains two integers u and v — the edges e_1 and e_2. Graph G and edges e_1, e_2 must satisfy all the constraints and properties described above.

*To flush, use:

- `fflush(stdout)` or `cout.flush()` in C++;
- `sys.stdout.flush()` in Python;
- see the documentation for other languages.

interaction log

```
< 2
< 4 2
< 1 2
< 3 4
  > ? 1 3 1 3 4
< 0
  > ? 2 1 3
< 1
  > ! 1 3
< 5 3
< 1 2
< 2 3
< 4 5
  > ? 2 1 4
< 1
  > ! 1 4
```

In the first test case, the hidden edges are $(1, 3)$ and $(1, 4)$. The first query selects vertices 1, 3, and 4, which form a triangle in G', so the response is 0. The second query confirms that edge $(1, 3)$ exists, and we report this hidden edge.

In the second test case, the hidden edges are $(1, 4)$ and $(3, 5)$. The query confirms that edge $(1, 4)$ exists, and we report this hidden edge.

F. Fun Times

2 seconds, 256 megabytes

For a set $\mathcal{A}$ of integers from 0 to $n - 1$, define the value $f(\mathcal{A})$ as follows. The set undergoes $n - 1$ steps: at step i ($1 \le i \le n - 1$), each element x of the current set is replaced with $(x + 1) \bmod (n - i)$. Note that all elements are replaced simultaneously and are kept in the set only once, so after a step the size of the set may decrease.

Let $\mathcal{A}_i$ denote the set obtained after the i-th step. Then $f(\mathcal{A})$ is the sum of the values of mex^* of all obtained sets:

$$f(\mathcal{A}) = \operatorname{mex}(\mathcal{A}_1) + \operatorname{mex}(\mathcal{A}_2) + \ldots + \operatorname{mex}(\mathcal{A}_{n-1}).$$

Dima has a set $\mathcal{S}$ consisting of distinct integers from 0 to $n - 1$.

Dima wants to find the sum of $f(\mathcal{S}')$ over all sets $\mathcal{S}'$ such that $\mathcal{S} \subseteq \mathcal{S}' \subseteq \{0, 1, \ldots, n - 1\}$. In other words, he wants the sum of f over all supersets of $\mathcal{S}$ contained in $\{0, 1, \ldots, n - 1\}$. Please help him.

Since the answer can be very large, output it modulo $10^9 + 7$.

*The minimum excluded (MEX) of a collection of integers $c_1, c_2, \ldots, c_k$ is defined as the smallest non-negative integer x which does not occur in the collection c.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 10^4$). The description of the test cases follows.

The first line of each test case contains a single integer n ($1 \le n \le 5 \cdot 10^5$) — the length of the string.

The second line contains a binary string s of length n, representing the set $\mathcal{S}$. For each $0 \le i < n$, the integer i belongs to $\mathcal{S}$ if and only if the $(i + 1)$-th character of s is `1`.

It is guaranteed that the sum of n over all test cases does not exceed $5 \cdot 10^5$.

Output

For each test case, output one integer — the required sum modulo $10^9 + 7$.

input
4 5 10110 3 000 9 101010101 6 000111
output
30 14 306 78

In the first example, there are only 4 sets containing the given $\mathcal{S}$ as a subset: $\{0, 2, 3\}$, $\{0, 1, 2, 3\}$, $\{0, 2, 3, 4\}$, and $\{0, 1, 2, 3, 4\}$.

For the set $\mathcal{A} = \{0, 2, 3\}$, the sequence of transformations is as follows:

$$\{0, 2, 3\} \to \{0, 1, 3\} \to \{1, 2\} \to \{0, 1\} \to \{0\}$$

The values of mex of the obtained sets are $2, 0, 2, 1$, respectively, so:

$$f(\{0, 2, 3\}) = 2 + 0 + 2 + 1 = 5.$$

For the set $\mathcal{A} = \{0, 1, 2, 3\}$:

$$\{0, 1, 2, 3\} \to \{0, 1, 2, 3\} \to \{0, 1, 2\} \to \{0, 1\} \to \{0\}$$

$$f(\{0, 1, 2, 3\}) = 4 + 3 + 2 + 1 = 10.$$

For the set $\mathcal{A} = \{0, 2, 3, 4\}$:

$$\{0, 2, 3, 4\} \to \{0, 1, 3\} \to \{1, 2\} \to \{0, 1\} \to \{0\}$$

$$f(\{0, 2, 3, 4\}) = 2 + 0 + 2 + 1 = 5.$$

For the set $\mathcal{A} = \{0, 1, 2, 3, 4\}$:

$$\{0, 1, 2, 3, 4\} \to \{0, 1, 2, 3\} \to \{0, 1, 2\} \to \{0, 1\} \to \{0\}$$

$$f(\{0, 1, 2, 3, 4\}) = 4 + 3 + 2 + 1 = 10.$$

Thus, the final answer is the sum over these four sets:

$$5 + 10 + 5 + 10 = 30.$$

In the second test case, $\mathcal{S}$ is empty, so all 8 subsets of $\{0, 1, 2\}$ are considered. The value of f is 0 for the empty set, 1 for each of $\{0\}$, $\{2\}$, and $\{0, 2\}$, 2 for $\{1\}$, and 3 for each of the remaining three sets. Thus, the answer is $0 + 3 \cdot 1 + 2 + 3 \cdot 3 = 14$.

Codeforces (c) Copyright 2010-2026 Mike Mirzayanov
The only programming contests Web 2.0 platform